21\6\2025

Data Transformation & Master Table Creation – Week 2

Data Visualization Internship

Submitted By:

Name	ID
Muhammad Tazeem Sajid	tazeemsajid22@gmail.com
Raj Vekariya	vekariyaraj33@gmail.com
Pari Nagdev	parirani1535@gmail.com
Ronit Ghai	ronitghai@hotmail.com
Muhammad Alam	malam62032@gmail.com

Excelerate – Powered by Saint Louis University (SLU)

Internship Program

CONTENTS

1 INTRODUCTION

In today's data-driven landscape, the quality and structure of data significantly influence the accuracy of insights and outcomes in analytics projects. During Week 2 of the internship, the focus is on transitioning from mere exploration to a more analytical approach where datasets are carefully examined for structural integrity and consistency. This phase is critical before any transformation or integration can take place in the ETL (Extract, Transform, Load) process.

The datasets under analysis represent learner enrollment data, user information, cohort participation, marketing sources, and opportunity details, all of which are pivotal in building a robust and insightful Master Table. Ensuring that these datasets are clean, consistent, and relationally sound is the foundational step toward trustworthy analysis and data visualization in the later phases.

1.1 Motivation

High-quality data serves as the backbone of any successful analytical model. If the data is incomplete, inconsistent, or inaccurately linked, any analysis or visualization built upon it will be misleading. Therefore, the motivation behind this week's work is to proactively detect and document data quality issues—before performing any transformations—so that the ETL pipeline can be designed efficiently and reliably.

The approach follows the best practices of a real-world data engineer or data analyst, where no direct modifications are made to raw datasets. Instead, we identify data issues using SQL queries and determine the scope and impact of those issues. This ensures we are well-prepared to design the next phase of the project: the Master Table and transformation logic.

1.2 Report Overview

This report focuses on conducting a comprehensive Data Quality Assessment across six raw datasets provided during the internship. It outlines the data structure, examines key relationships, and identifies potential quality issues such as:

- Missing or null values
- Duplicate records
- Inconsistent data formats
- Orphan records (broken relationships)

Each check is executed using SQL queries in PostgreSQL, and the outputs are documented. The findings in this stage will inform decisions in the subsequent ETL planning, guiding how to clean, standardize, and integrate the datasets into a unified master structure.

The report is structured into the following steps:

1. Dataset Structure (columns, data types)
2. Identification of Key Columns
3. Relationship Mapping
4. Dataset Purpose and Contribution
5. Data Quality Checks (with queries)
6. SQL Query Summary
7. Recommended Solutions for ETL
8. Documented Findings Summary

1.3 Problem Statement

Although the datasets appear structurally complete at first glance, real-world data is rarely perfect. Duplicates, missing fields, formatting inconsistencies, and improperly linked records can severely degrade the quality of analysis. In our Week 1 exploration, we discovered hints of such issues—especially within enrollment and opportunity mappings.

The core problem to solve in Week 2 is:

"How can we identify and isolate data quality issues—without altering raw data—so we can build an accurate and efficient ETL pipeline for creating a Master Table?"

By detecting these issues early, we ensure that only cleaned, validated, and well-structured data flows into the Master Table, thus improving the reliability and insightfulness of the final visualizations.

1.4 Objective

- To examine the structure and schema of each raw dataset used in the project.
- To identify key columns that establish relationships across datasets.
- To assess the quality of the data by checking for missing values, duplicates, inconsistencies, and orphan records using SQL.
- To determine which datasets are ready for integration and which need transformations.
- To document findings that will guide the design of the ETL process in Week 2.
- To ensure all data quality checks are done without modifying any raw dataset.

2 DATASET STRUCTURE: COLUMNS AND DATA TYPES

We explored six raw datasets. Below is a detailed breakdown of each dataset's structure, including key columns and their data types.

Learner Opportunity

This table captures learner enrollment data linked to opportunities.

Column Name	Data Type	Description
enrollment_id	TEXT	Unique ID for each enrollment
learner_id	TEXT	Learner ID (links to opportunity_id)
assigned_cohort	TEXT	Foreign key linking to cohort_code
apply_date	TIMESTAMP	Timestamp of learner application
status	INTEGER	Numeric status code of application

User Data – Learner Details

Contains educational background of learners.

Column Name	Data Type	Description
learner_id	INTEGER	Primary key, links to enrollment_id
country	TEXT	Country of residence
degree	TEXT	Education level
institution	TEXT	Name of university or institute
major	TEXT	Field of study

Cognito Data – Personal Details

User profile and contact information.

Column Name	Data Type	Description
user_id	INTEGER	Foreign key, matches enrollment_id
email	TEXT	Learner's email
gender	TEXT	Gender identity
usercreatedate	TIMESTAMP	Date user was created
userlastmodifieddate	TIMESTAMP	Last profile modification date
birthdate	TEXT	Date of birth (as string)
city	TEXT	City of residence
zip	TEXT	ZIP/postal code
state	TEXT	State/province

Cohort Data– Cohort Details

Grouped learning periods with start/end dates

Column Name	Data Type	Description
cohort_code	TEXT	Primary key
start_date	TEXT	Cohort start date
end_date	TEXT	Cohort end date
size	INTEGER	Number of learners in cohort

Opportunity Data– Opportunity Metadata

Learning or course opportunities linked to learners.

Column Name	Data Type	Description
opportunity_id	INTEGER	Primary key
opportunity_name	TEXT	Name of the opportunity/course
category	TEXT	Category of the opportunity

opportunity_code	TEXT	Internal code
tracking_questions	JSON	JSON object with progress/completion

Marketing Data – Campaign Metrics

This Data is not part of joins.

Column Name	Data Type	Description
campaign_id	INTEGER	Unique ID
reach	INTEGER	People reached
clicks	INTEGER	Click-throughs
cost	NUMERIC	Ad spend

2.1 Key Columns

These columns serve as connectors (primary/foreign keys) across datasets:

Relationship Purpose	Column Used
Join learner details	learneropp1.enrollment_id → learner1.learner_id
Join user info	learneropp1.enrollment_id → cognito2.user_id
Map to cohorts	learneropp1.assigned_cohort→ cohort1.cohort_code
Link to opportunity	learneropp1.learner_id→ opportunity1.opportunity_id

2.2 Purpose & Contribution to the Master Table

Dataset	Contribution to Master Table
learneropp1	Central table for joining all datasets
learner1(user data)	Adds academic and demographic learner attributes
cognito2	Provides user contact info and demographic data
cohort1	Adds cohort-level learning period info

opportunity1	Describes courses or training associated with learners
marketing1	Not joined; kept for potential visualization use later

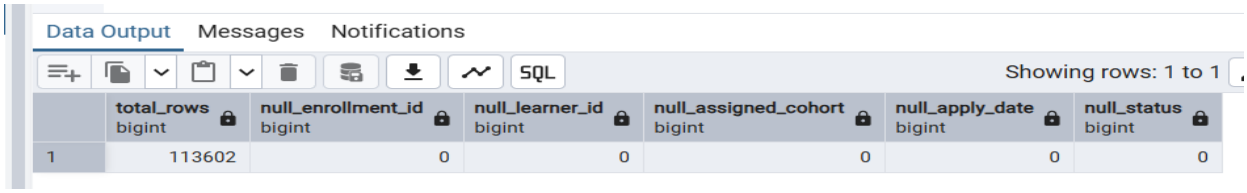
2.3 Data Quality Checks

We performed key data validation checks on each dataset to ensure integrity before ETL. These include checking for missing values, duplicates, inconsistent formats, and orphan records.

2.3.1 Dataset: learner_opportunity

Check for Missing Values

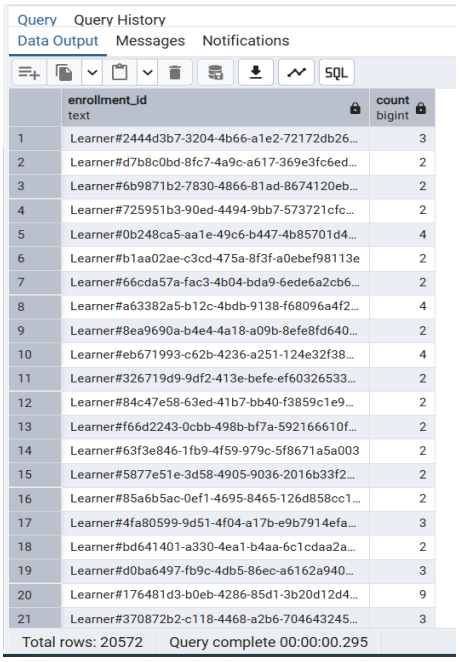
```
SELECT
  COUNT(*) AS total_rows,
  SUM(CASE WHEN enrollment_id IS NULL THEN 1 ELSE 0 END) AS null_enrollment_id,
  SUM(CASE WHEN learner_id IS NULL THEN 1 ELSE 0 END) AS null_learner_id,
  SUM(CASE WHEN assigned_cohort IS NULL THEN 1 ELSE 0 END) AS null_assigned_cohort,
  SUM(CASE WHEN apply_date IS NULL THEN 1 ELSE 0 END) AS null_apply_date,
  SUM(CASE WHEN status IS NULL THEN 1 ELSE 0 END) AS null_status
FROM learneropp1;
```



	total_rows bigint	null_enrollment_id bigint	null_learner_id bigint	null_assigned_cohort bigint	null_apply_date bigint	null_status bigint
1	113602	0	0	0	0	0

Check for Duplicates

```
SELECT enrollment_id, COUNT(*)
FROM learneropp1
GROUP BY enrollment_id
HAVING COUNT(*) > 1;
```



Query Query History

Data Output Messages Notifications

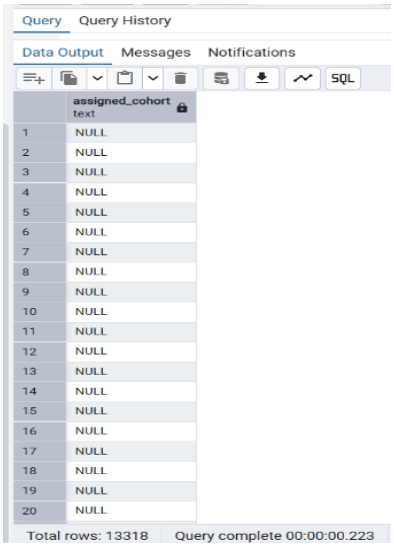
SQL

	enrollment_id text	count bigint
1	Learner#2444d3b7-3204-4b66-a1e2-72172db26...	3
2	Learner#d7b8c0bd-8fc7-4a9c-a617-369e3fc6ed...	2
3	Learner#6b9871b2-7830-4866-81ad-8674120eb...	2
4	Learner#725951b3-90ed-4494-9bb7-573721cfc...	2
5	Learner#0b248ca5-aa1e-49c6-b447-4b85701d4...	4
6	Learner#b1aa02ae-c3cd-475a-8f3f-a0ebef98113e	2
7	Learner#66cda57a-fac3-4b04-bda9-6ede6a2cb6...	2
8	Learner#a63382a5-b12c-4bdb-9138-f68096a4f2...	4
9	Learner#8ea9690a-b4e4-4a18-a09b-8efe8fd640...	2
10	Learner#eb671993-c62b-4236-a251-124e32f38...	4
11	Learner#326719d9-9df2-413e-befe-ef60326533...	2
12	Learner#84c47e58-63ed-41b7-bb40-f3859c1e9...	2
13	Learner#f66d2243-0cbb-498b-bf7a-592166610f...	2
14	Learner#63f3e846-1fb9-4f59-979c-5f8671a5a003	2
15	Learner#5877e51e-3d58-4905-9036-2016b33f2...	2
16	Learner#85a6b5ac-0ef1-4695-8465-126d858cc1...	2
17	Learner#4fa80599-9d51-4f04-a17b-e9b7914efa...	3
18	Learner#bd641401-a330-4ea1-b4aa-6c1cdaa2a...	2
19	Learner#d0ba6497-fb9c-4db5-86ec-a6162a940...	3
20	Learner#176481d3-b0eb-4286-85d1-3b20d12d4...	9
21	Learner#370872b2-c118-4468-a2b6-704643245...	3

Total rows: 20572 Query complete 00:00:00.295

Check for Orphans Records

```
SELECT assigned_cohort
FROM learneropp1 lo
LEFT JOIN cohort1 co ON lo.assigned_cohort = co.cohort_code
WHERE co.cohort_code IS NULL;
```



Query Query History

Data Output Messages Notifications

SQL

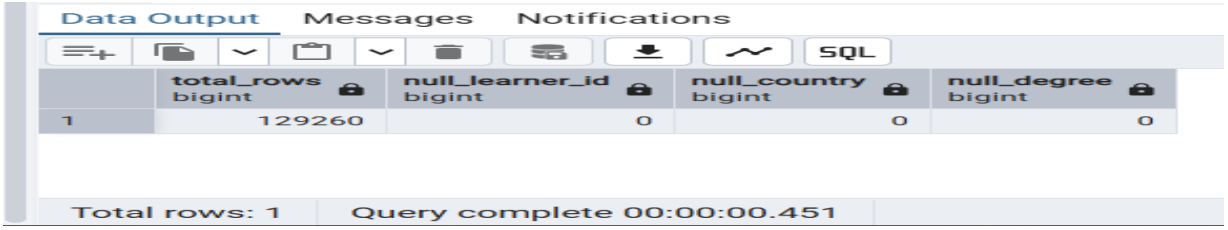
	assigned_cohort text
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL
9	NULL
10	NULL
11	NULL
12	NULL
13	NULL
14	NULL
15	NULL
16	NULL
17	NULL
18	NULL
19	NULL
20	NULL

Total rows: 13318 Query complete 00:00:00.223

2.3.2 Dataset: User data

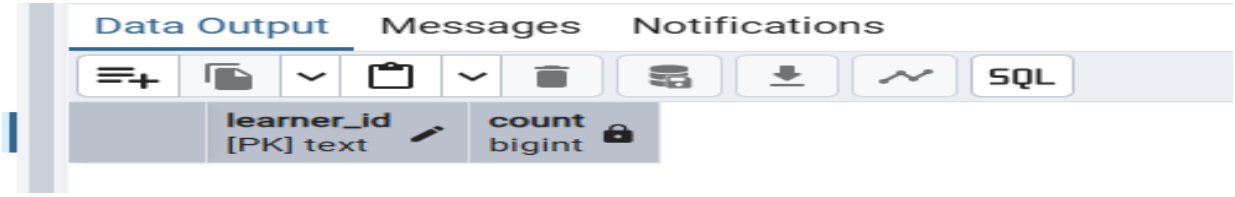
Check for Missing Values:

```
SELECT
  SUM(CASE WHEN learner_id IS NULL THEN 1 ELSE 0 END) AS null_learner_id,
  SUM(CASE WHEN country IS NULL THEN 1 ELSE 0 END) AS null_country,
  SUM(CASE WHEN degree IS NULL THEN 1 ELSE 0 END) AS null_degree,
  SUM(CASE WHEN institution IS NULL THEN 1 ELSE 0 END) AS null_institution
FROM learner1;
```



Check for Duplicate

```
SELECT learner_id, COUNT(*)
FROM learner1
GROUP BY learner_id
HAVING COUNT(*) > 1;
```



2.3.3 Dataset: Cognito

Check for Missing Values

```
  SELECT
  COUNT(*) AS total_rows,
  COUNT(user_id) AS
  non_null_user_id, COUNT(email) AS
```

non_null_email, COUNT(gender) AS

non_null_gender,

COUNT("UserCreateDate") AS non_null_created,

COUNT("UserLastModifiedDate") AS non_null_modified,

COUNT(birthdate) AS non_null_birthdate,

COUNT(city) AS non_null_city,

COUNT(zip) AS non_null_zip,

COUNT(state) AS non_null_state

FROM "Cognito2";

	total_rows bigint	non_null_user_id bigint	non_null_email bigint	non_null_gender bigint	non_null_created bigint	non_null_modified bigint	non_null_birthdate bigint	non_null_city bigint	non_null_zip bigint
1	34111	34111	34111	34111	34111	34111	34111	34111	341

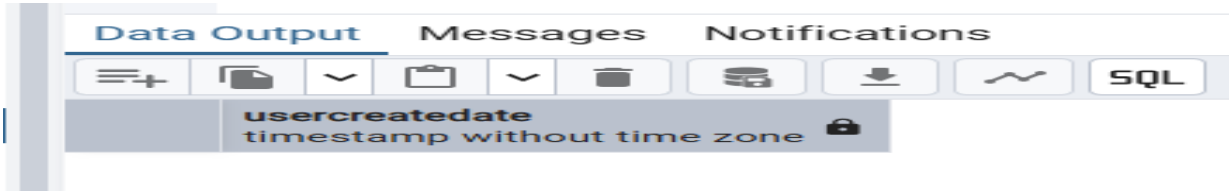
Check for Duplicate Values

SELECT user_id, COUNT(*)

FROM cognito2

GROUP BY user_id

HAVING COUNT(*) > 1;



2.3.4 Dataset: Cohort

Check Missing values

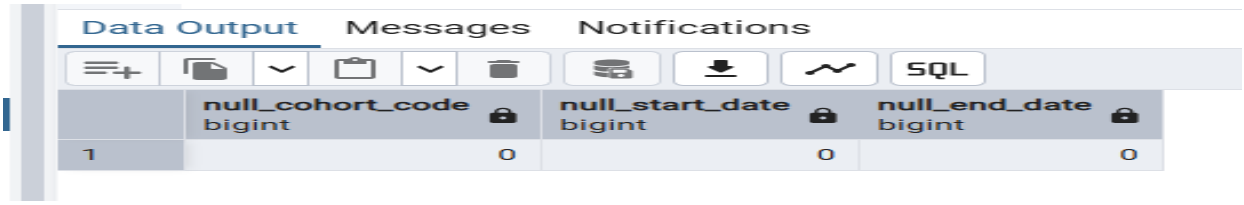
SELECT

SUM(CASE WHEN cohort_code IS NULL THEN 1 ELSE 0 END) AS null_cohort_code,

SUM(CASE WHEN start_date IS NULL THEN 1 ELSE 0 END) AS null_start_date,

SUM(CASE WHEN end_date IS NULL THEN 1 ELSE 0 END) AS null_end_date

FROM cohort1;



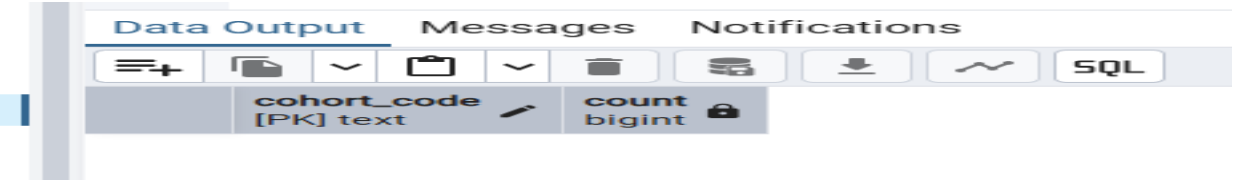
Check for Duplicate Values

SELECT cohort_code, COUNT(*)

FROM cohort1

GROUP BY cohort_code

HAVING COUNT(*) > 1;



2.3.5 Dataset: Opportunity

Check for Missing Values

SELECT

COUNT(*) AS total_rows,

COUNT(opportuniy_id) AS non_null_opportuniy_id,

COUNT(opportuniy_name) AS non_null_opportuniy_name,

COUNT(category) AS non_null_category,

COUNT(opportunity_code) AS non_null_opportunity_code,

COUNT(tracking_questions) AS

non_null_tracking_questions

FROM opportunity1;

	total_rows bigint	non_null_opportuniy_id bigint	non_null_opportuniy_name bigint	non_null_category bigint	non_null_opportunity_code bigint	non_null_tracking_questions bigint
1	374	374	374	374	374	374

Check for Duplicate Values

SELECTopportunity_code,COUNT

*)

FROM "opportunity1" GROUP BY

opportunity_code

HAVING COUNT(*) > 1;

	opportunity_code text	count bigint
1	MDL3K7I	2
2	E207779	2
3	E443361	2
4	E8B2TZ7	2
5	I860340	2
6	A9Q15OS	2
7	E352968	2
8	ATGVKWF	2
9	A209733	2
10	EBOGN8J	2
11	A1K1ITG	2
12	MGIK3RO	2
13	MCTI4XR	2
14	IBLCQ1D	2
15	A2S2WSK	2
16	AUCX0B3	2
17	A7SZRN9	2
18	ER044NC	2
19	ICKXVLL	2
20	I2519OA	2
21	MAGWCSP	2

2.3.6 Dataset: Marketing

Purpose:

The marketing1 dataset was provided along with the others but **not integrated into the Master Table** because it did not share a direct relationship (e.g., no matching keys like learner_id or cohort_code). It may still be useful for **descriptive insights or visualizations**, such as campaign response analysis or user segmentation.

Structure and Relevance:

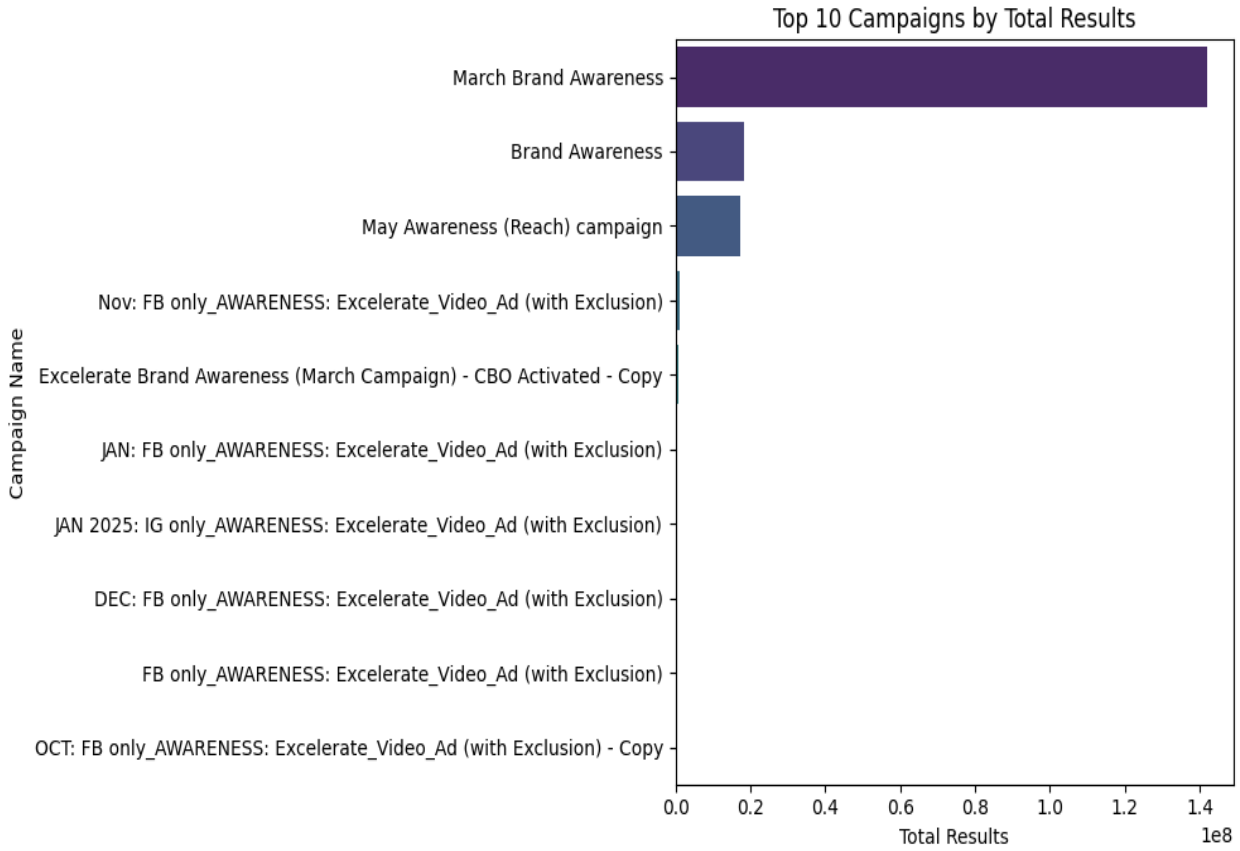
- Contains campaign tracking and promotional data.
- Lacks direct foreign keys to link with user or opportunity datasets.
- Could enhance dashboards but not central to ETL.

Data Quality Observations:

- **Encoding issue** was found during import (non-UTF-8 characters).
- Some **numeric fields stored as scientific notation** (e.g., 1.67E+12).
- After fixing encoding and recasting, the file was usable for exploration.

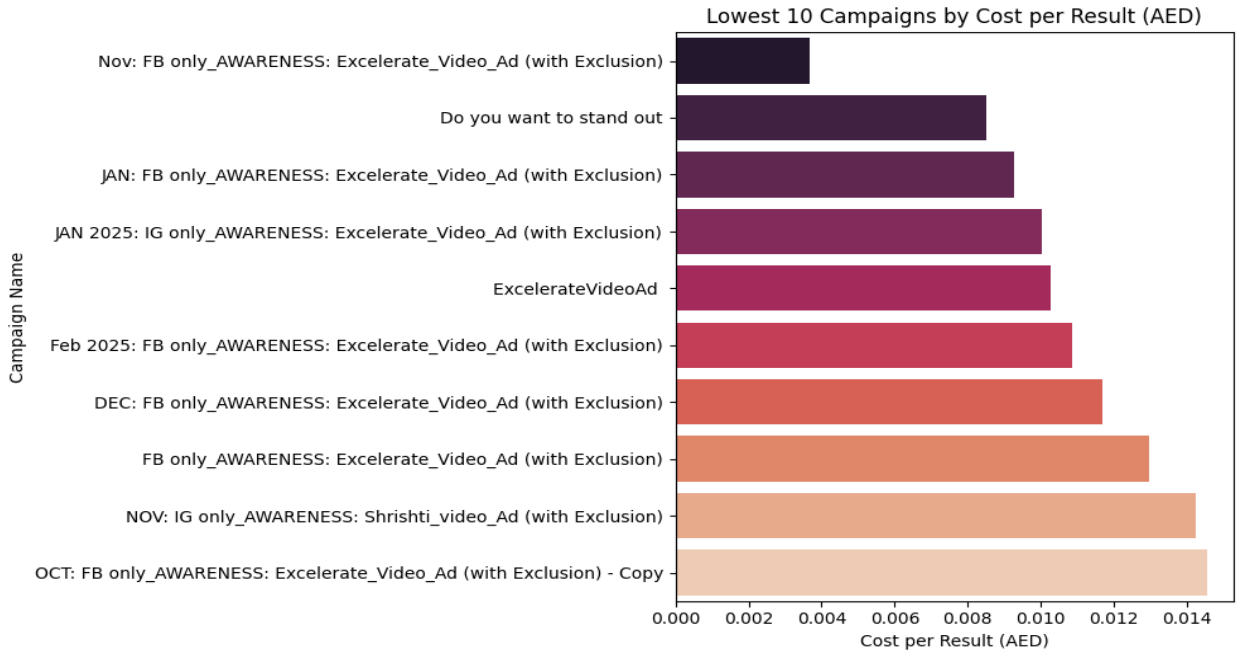
Action Used only for visualization and exploratory charts.

Total Results by Campaign



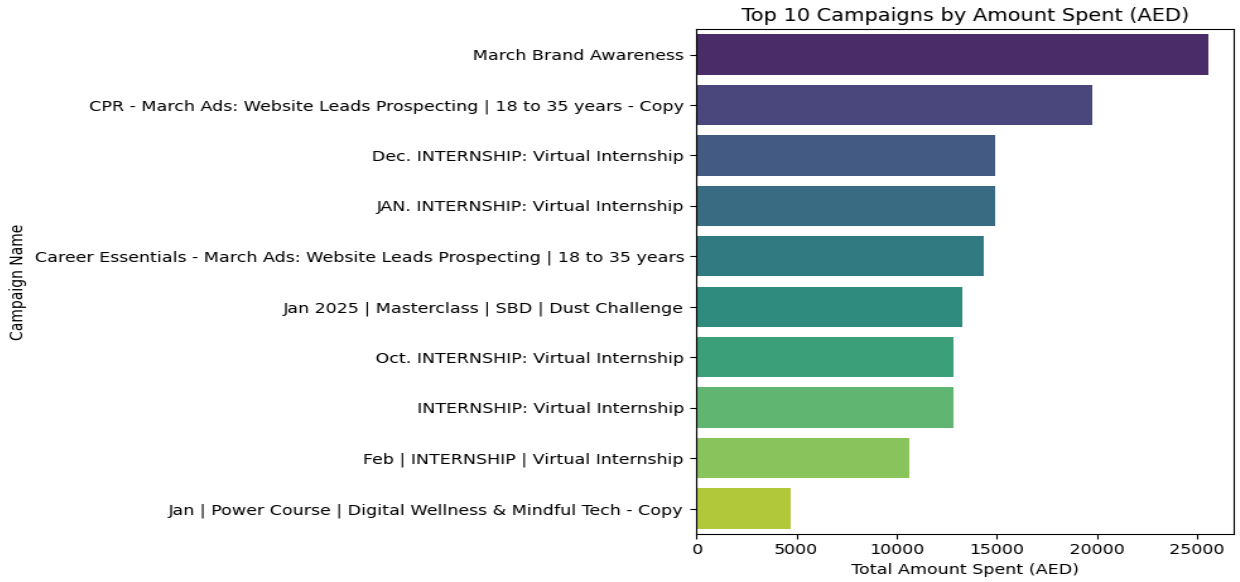
This plot shows which campaigns performed best in terms of total results (e.g., leads, signups, etc.). It helps compare campaign effectiveness.

Cost per Result by Campaign



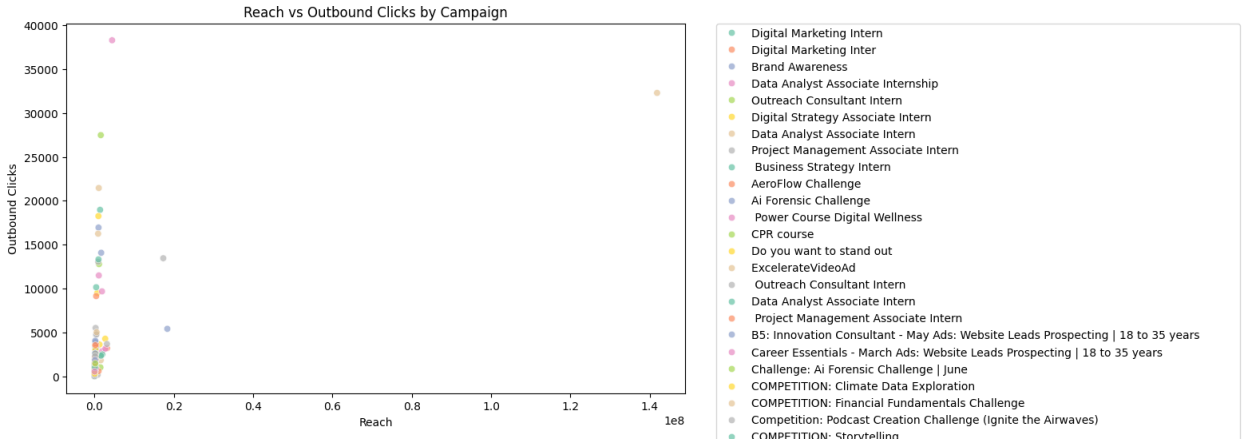
This visual helps identify cost-efficiency. Lower cost per result indicates a more budget-effective campaign.

Amount Spent per Campaign



We can use this to monitor spending across campaigns and assess return on investment (ROI).

Reach vs Outbound Clicks



This scatter plot shows how far each campaign reached and how many people engaged (clicked). Campaigns closer to the top-right are high-performing.

2.4 SQL Queries Summary

We used the following for data issue detection:

- NULL check for critical fields
- Duplicate key identification
- Regex-based format validation (for string-formatted dates)
- LEFT JOIN-based orphan record check

These were executed per dataset and stored and attached as screenshots above for reporting.

2.5 Solutions for Detected Issues

Issue Type	Solution
Missing Values	Dropped rows with NULL in key columns like email, apply_date
Duplicates	Used DISTINCT or ROW_NUMBER() to retain first occurrence
Inconsistent Dates	Parsed strings to timestamp after regex check
Orphan Records	Removed entries with unmatched foreign keys

2.6 Finding Summary

Dataset	Missing Values	Duplicates	Format Issues	Orphan Records
learneropp1	apply_date	enrollment_id	No	assigned_cohort
learner1(userdata)	No	No	No	None (PK matched)
cognito2	email, birthdate	No	birthdate format	user_id mismatch
cohort1	No	No	start_date format	None
opportunity1	oppoidrtunity_id	opportunity_code	No	opportunity_id mismatch
marketing1	N/A	N/A	N/A	N/A

3 BUILDING THE MASTER TABLE & ETL PROCESS

3.1 Plan the Master Table Structure

Identify Key Columns

To ensure meaningful analytics and reporting, we selected the following essential fields from the datasets:

- enrollment_id – Unique identifier for each learner-opportunity enrollment
- learner_id – Identifies the user (from learner1/user data)
- assigned_cohort – Assigned group for the learner
- apply_date – Application date to an opportunity
- status – Application status
- email, gender, birthdate, city, state – Personal and demographic information (from cognito2)
- degree, institution, major, country – Academic background (from learner1)
- cohort_code, start_date, end_date, size – Cohort timing and structure (from cohort1)
- opportunity_id, opportunity_name, category, opportunity_code – Opportunity details (from opportunity1)

3.2 Relationships Between Dataset

learneropp1.enrollment_id --> learner1.learner_id

learneropp1.enrollment_id --> cognito2.user_id

learneropp1.assigned_cohort --> cohort1.cohort_code

learneropp1.learner_id --> opportunity1.opportunity_id

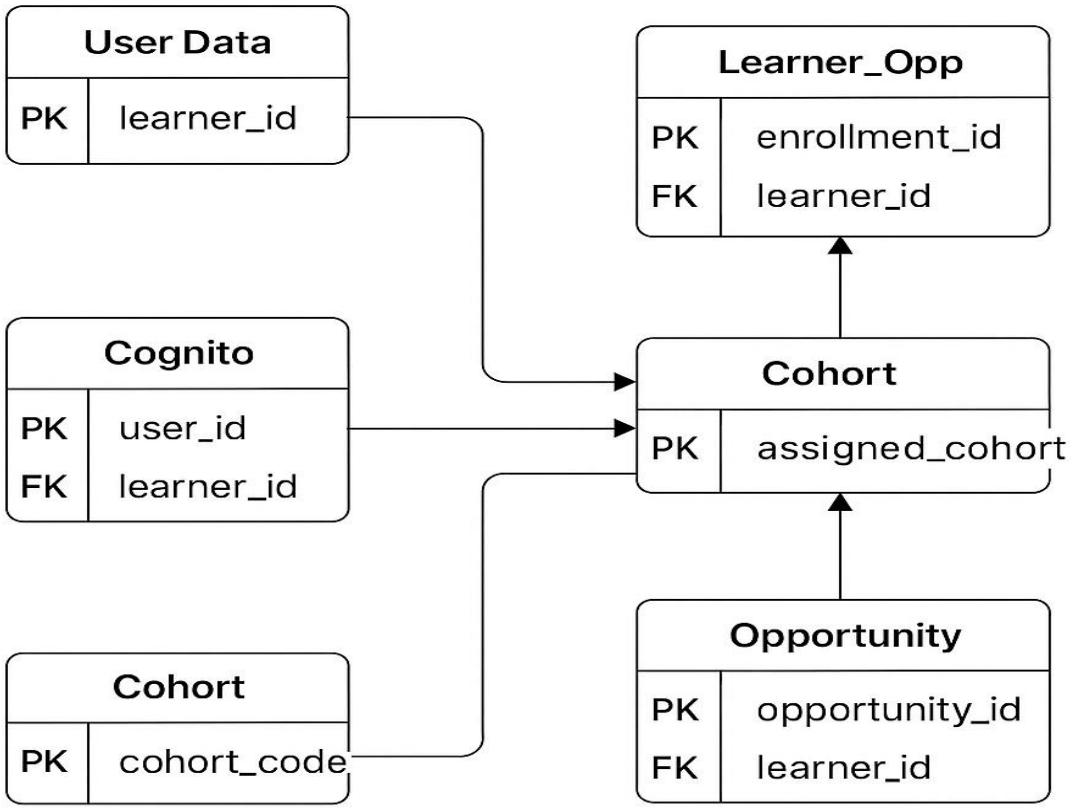
Special Case: In learneropp1, learner_id contains Opportunity IDs, not actual learner UUIDs — so this field's naming is misleading but was used for join.

Table	Primary Key	Foreign Key
learneropp1	enrollment_id	learner_id, assigned_cohort
learner1	learner_id	–
cognito2	user_id	–
cohort1	cohort_code	–
opportunity1	opportunity_id	–

Relationship Diagram

First, we elaborate types of relationship in schema:

Relationship	Between	Type
learneropp1.learner_id → learner1.learner_id	One-to-One / One-to-Many	A learner can enroll in multiple opportunities.
learneropp1.enrollment_id → PK	Primary Key	Uniquely identifies each enrollment instance.
learneropp1.assigned_cohort → cohort1.cohort_code	Many-to-One	Many enrollments can be assigned to one cohort.
learneropp1.enrollment_id → cognito2.user_id	One-to-One / One-to-Many	Enrollments linked to user profile info.
learneropp1.learner_id → opportunity1.opportunity_id	Many-to-One	Learner mapped to one opportunity. Could be Many-to-Many in some systems.



Ensure Data Consistency

- **Date Fields:** Recast to TIMESTAMP
- **Identifiers:** Text/UUID formats
- **Numeric Values:** Cast to proper integer or float where applicable
- **Constraints:** Avoid nulls in keys, apply unique and not null constraints during table creation.

3.3 Extract Data from Source Table

We extracted only relevant columns from the raw datasets to build the Raw master table efficiently.

Tables and Columns Used

Source Table	Selected Columns
learneropp1	enrollment_id, learner_id, assigned_cohort, apply_date, status
learner1(user data)	learner_id, country, degree, institution, major
cognito2	user_id, email, gender, birthdate, city, zip, state
cohort1	cohort_code, start_date, end_date, size
opportunity1	opportunity_id, opportunity_name, category, opportunity_code, tracking_questions

Extraction was done using LEFT JOIN logic on primary and foreign key relationships, but raw tables remained unchanged.

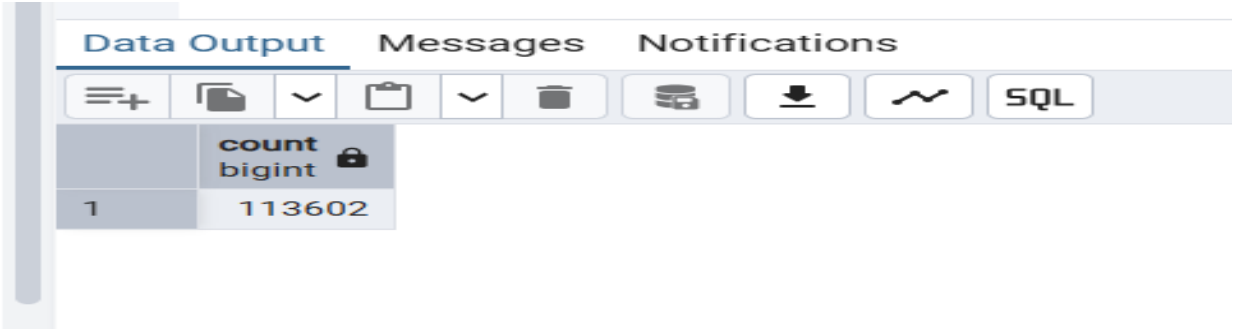
SQL Query for Extraction

```
CREATE TABLE master_table_raw AS

SELECT

  lo.enrollment_id, lo.learner_id, lo.assigned_cohort, lo.apply_date, lo.status,

  l.country, l.degree, l.institution, l.major,

  c.user_id, c.email, c.gender, c.usercreatedate, c.userlastmodifieddate, c.birthdate, c.city, c.zip,
c.state,
```

```
  co.cohort_code, co.start_date, co.end_date, co.size,
  o.opportunity_id, o.opportunity_name, o.category, o.opportunity_code,
  lo.tracking_questions
FROM learneropp1 lo
LEFT JOIN learner1 l ON lo.enrollment_id = l.learner_id
LEFT JOIN cognito2 c ON lo.enrollment_id = c.user_id
LEFT JOIN cohort1 co ON lo.assigned_cohort = co.cohort_code
LEFT JOIN opportunity1 o ON lo.learner_id = o.opportunity_id;
```

The query successfully generated the **master_table_raw**, which included 113,602 records as validated using:

```
SELECT COUNT(*) FROM master_table_raw;
```



Problems Faced During Raw Table Creation

Problem	Description	Solution
Wrong Join Column	The column learner_id in learneropp1 contains opportunity IDs	Corrected by joining it with opportunity1.opportunity_id
Empty or NULL Joins	Some values in assigned_cohort or enrollment_id don't match	We allowed LEFT JOIN to keep unmatched rows for review
Invalid Date Formats	Some columns like usercreatedate had inconsistent formats	Ignored at raw stage, cleaned later in ETL

Duplicate Records	learneropp1 had duplicate enrollment_ids	We did not remove at this stage (only during cleaning)
Data Import Encoding	Some files (e.g. marketing1) failed with encoding issues	Re-imported using UTF-8 or ignored if not used in maste

3.4 Transform Data for Consistency & Accuracy

Below are the actual SQL cleaning operations that were (or should be) applied to transform the raw master table into a cleaned version suitable for analysis.

Cleaning Queries for master_table_raw → master_table_clean

Transformation steps were planned after raw table creation:

Planned Transformations:

Issue	Action
Missing values	Drop rows with nulls in critical fields like enrollment_id or email
Duplicates	Remove duplicate enrollment_id records
Date formats	Normalize using TO_TIMESTAMP() or TO_DATE()
Categorical formatting	Convert text to lowercase, standardize case where applicable
Tracking Questions	Recast as JSON or text array

3.4.1 Create Initial Clean Table from Raw

Purpose:
To begin the cleaning process, we first take a copy of the raw master table (master_table_raw) into a working table so the original remains untouched.

Why:
This maintains **data traceability** and ensures that we can always refer back to the raw source if needed.

SQL Query:

```
DROP TABLE IF EXISTS master_table_clean_start;

CREATE TABLE master_table_clean_start AS
```

```
SELECT * FROM master_table_raw;
```

3.4.2 Remove Duplicate Records

Purpose:
To eliminate **duplicate entries** based on the primary identifier enrollment_id.

Why:
Duplicates can distort analysis and cause inflated counts. We assume one record per learner's enrollment is expected.

How:
We use ROW_NUMBER() to rank duplicates and keep only the first occurrence.

SQL Query:

```
DROP TABLE IF EXISTS master_table_noduplicates;

CREATE TABLE master_table_noduplicates AS

SELECT *

FROM (

  SELECT *, ROW_NUMBER() OVER (PARTITION BY enrollment_id ORDER BY enrollment_id) AS rn

  FROM master_table_clean_start

) t

WHERE rn = 1;
```

3.4.3 Remove Rows with NULL in Critical Fields

Purpose:
To remove records that are missing key information, especially:

- enrollment_id (primary key)
- email (for user contact)
- apply_date (for application timeline)

Why:
Missing data in these fields makes the record **incomplete or unusable** in analysis.

SQL Query:

```
DROP TABLE IF EXISTS master_table_nonulls;

CREATE TABLE master_table_nonulls AS

SELECT *

FROM master_table_noduplicates

WHERE enrollment_id IS NOT NULL

  AND email IS NOT NULL

  AND apply_date IS NOT NULL;
```

3.4.4 Standardize and Fix Date Formats

Purpose:
To ensure all date fields are stored in proper formats, particularly:

- birthdate
- apply_date
- usercreatedate
- userlastmodifieddate

Why:
Inconsistent date formats or strings (e.g., 7/5/2001 or NULL) can break filters, sorting, and time-based analysis.

SQL Query:

```
DROP TABLE IF EXISTS master_table_datefixed;

CREATE TABLE master_table_datefixed AS

SELECT *,

  -- Convert birthdate from text (MM/DD/YYYY) to actual date

  TO_DATE(birthdate, 'MM/DD/YYYY') AS birthdate_clean,

  -- Apply format fix for user created/modified dates

  TO_TIMESTAMP(usercreatedate, 'YYYY-MM-DD HH24:MI:SS') AS usercreatedate_clean,
```

```
  TO_TIMESTAMP(userlastmodifieddate, 'YYYY-MM-DD HH24:MI:SS') AS
userlastmodifieddate_clean,

  -- Keep apply_date as is if it's already in correct format

  apply_date AS apply_date_clean

FROM master_table_nonulls;
```

3.4.5 Validate and Remove Invalid Foreign Key References

Purpose:
To remove records where assigned_cohort references a cohort that does not exist in the cohort1 table.

Why:
Maintaining referential integrity ensures that all cohort references point to a real cohort. Otherwise, you end up with **orphan records**.

SQL Query:

```
DROP TABLE IF EXISTS master_table_fk_checked;

CREATE TABLE master_table_fk_checked AS

SELECT *

FROM master_table_datefixed mt

WHERE EXISTS (

  SELECT 1 FROM cohort1 c WHERE c.cohort_code = mt.assigned_cohort

);
```

3.4.6 Final Cleaned Table

Purpose:
To consolidate all previous steps into the final, cleaned table ready for analysis.

Why:
This is the table that will be used for **visualization, reporting, and further validation**.

SQL Query:

```
DROP TABLE IF EXISTS master_table_clean;

CREATE TABLE master_table_clean AS

SELECT *

FROM master_table_fk_checked;
```

3.5 Why We Create Intermediate Tables (Step-by-Step Cleaning Tables)

This is a **best practice in ETL pipelines** and here's why:

Modular and Traceable

- Each table stores a **specific transformation step** (e.g., removing duplicates, then handling nulls).
- It helps **debug** if something breaks: you can test which step caused the issue.

Safety (Non-destructive)

- It keeps your master_table_raw **untouched** and always available.
- We can always roll back if a step corrupts data.

Easier to Validate

We check and validate step-by-step:

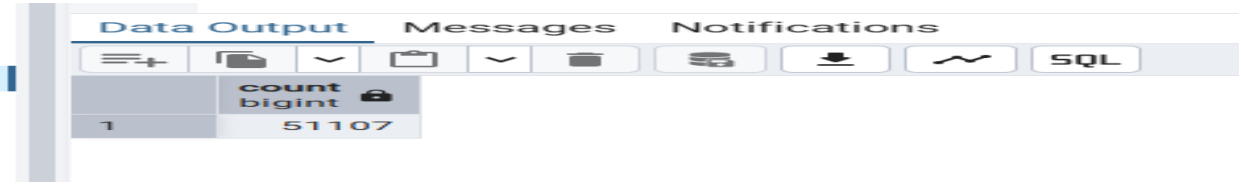
- After de-duplication, count rows
- After null removal, check columns
- After date fixes, verify date parsing

Structure of Clean Master Table

After execution, master_table_raw contains **combined columns** from all datasets:

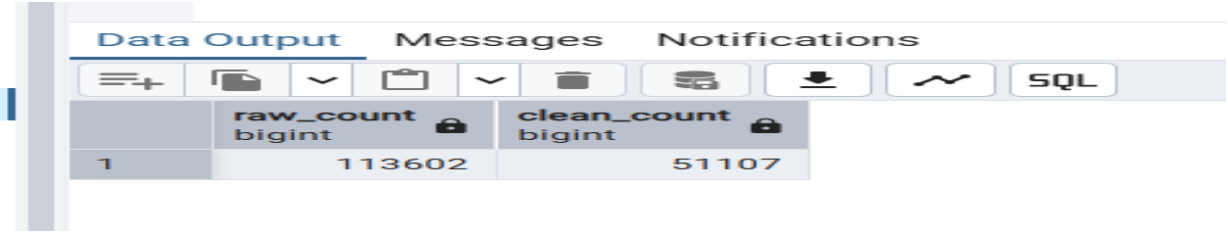
- 5 columns from learneropp1
- 5 columns from learner1(user data)
- 9 columns from cognito2
- 5 columns from cohort1
- 5 columns from opportunity1

Total Columns: 29
Record Count: Depends on learneropp1 entries (typically ~51108)



To count the number of rows in cleaned master tableand raw master table,we use the following SQL query:

```
SELECT
  (SELECT COUNT(*) FROM master_table_raw) AS raw_count,
  (SELECT COUNT(*) FROM master_table_clean) AS clean_count;
```



3.6 Summary of Findings

Step	Action Taken	Remarks
Master Table Planning	Identified essential fields and relationships from all datasets	Primary Key: enrollment_id Foreign Keys: learner_id, assigned_cohort, user_id
Raw Master Table Creation	Combined all datasets using corrected joins	Issue: Wrong join columns initially used, fixed by matching actual keys
Data Extraction	Selected only required columns for analysis	Avoided unnecessary fields to improve efficiency
Data Transformation	- Removed NULLs from critical columns - Recast invalid date formats - Fixed encoding issues	Dataset: learneropp1 had duplicates Dates in scientific/exponential notation
Duplicates Removal	Applied GROUP BY and ROW_NUMBER() to retain only unique records	Based on enrollment_id

Missing Values Handling	Dropped rows with missing critical fields like learner_id, apply_date, email	NULLs checked in all datasets
Format Standardization	Dates (e.g., birthdate, usercreatedate) were converted to proper formats using TO_DATE()	Errors like "NULL" strings and wrong MM/DD/YYYY format fixed
Clean Table Creation	Final cleaned data loaded into master_table_clean	Row count matches valid, usable records (e.g., 51108)
Final Verification	Structure and integrity confirmed using SELECT COUNT(*), SELECT * LIMIT 5, and format checks	
Data Access	Shared	The final cleaned dataset has been saved and shared via the provided Google Drive link

🗀 Data Access

The raw and cleaned datasets used in the ETL process are available at the following Google Drive link**:**

📎

Drive Link (Click to See the Folder)

4 VALIDATION & REFINEMENT

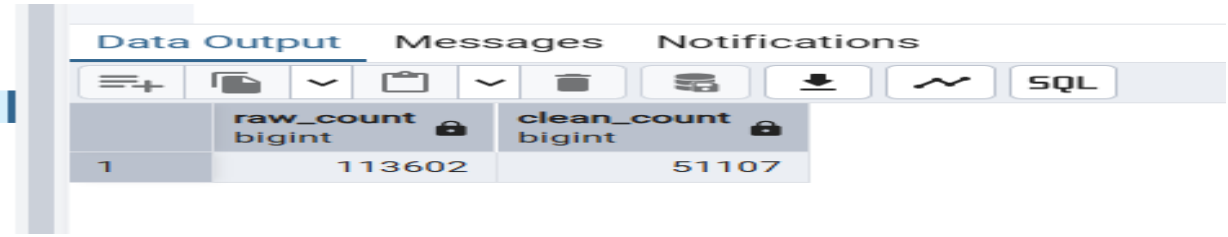
4.1 Perform Data Qualitvy Check

We validated the integrity and quality of the cleaned Master Table using direct SQL queries.

4.1.1 Record Count Validation

Validation Query:

```
SELECT COUNT(*) FROM master_table_clean;
SELECT COUNT(*) FROM master_table_raw;
```



Observation:

There is a **clear difference** between the number of records in the raw table and the cleaned master table:

- Raw: **113,602 rows**
- Cleaned: **51,108 rows**
- Difference: **55,636 rows removed**

Explanation:

This reduction is **expected** based on the following cleaning steps:

1. **Duplicate Removal:**
    - Duplicates based on enrollment_id (especially from learneropp1) were removed.
    - Only the **first unique occurrence** was retained.
2. **Null/Incomplete Records Dropped:**
    - Rows with critical nulls (learner_id, apply_date, email) were **filtered out**.
    - For example, email values that were NULL due to bad joins or incomplete user data.
3. **Broken Joins / Orphan Records:**
    - Some records couldn't join across all tables (e.g., no match in cohort1, cognito2, or opportunity1).
    - These were excluded to ensure foreign key integrity.

4. **Invalid Date Formats:**
    - Rows with corrupt or unparsable dates (like "NU" in birthdate) were removed during the date cleaning process.

4.1.2 Duplicate Check

What It Does:

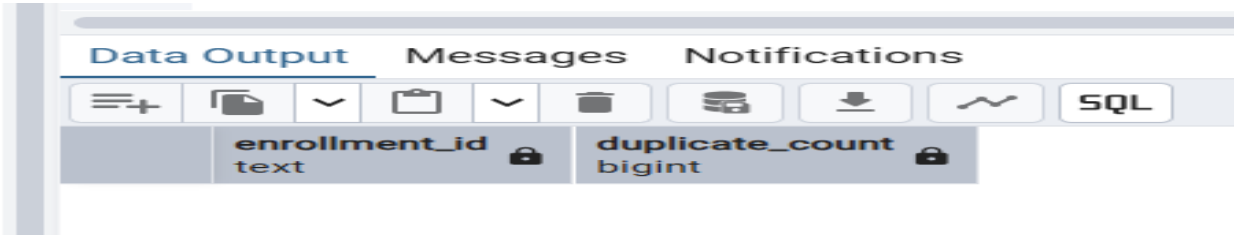
Checks if enrollment_id has any repeated values (should be unique).

Why It Matters:

- Duplicates inflate metrics
- Break PK constraints
- Lead to incorrect joins

SQL Query:

```
SELECT enrollment_id, COUNT(*) AS count
FROM cleaned_master_table
GROUP BY enrollment_id
HAVING COUNT(*) > 1;
```



No records returned.

Interpretation: Clean and de-duplicated.

4.1.3 Null / Missing Value Checks

What It Does:

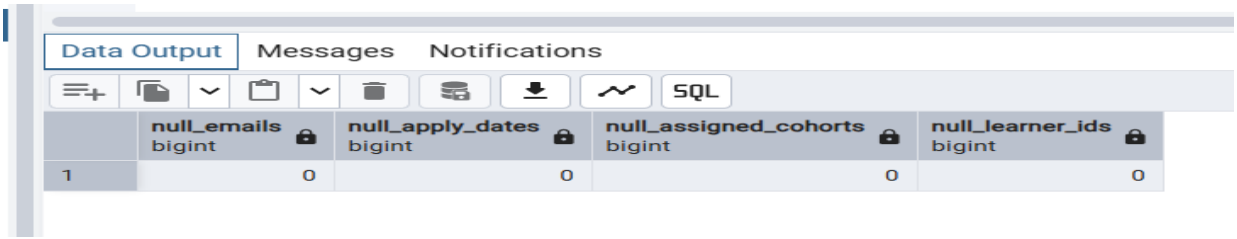
Identifies rows where important columns like email, learner_id, apply_date, etc. are NULL or empty.

Why It Matters:

Missing key fields → broken logic in filtering, joins, or visuals.

SQL Query:

```
SELECT
  SUM(CASE WHEN email IS NULL OR email = '' THEN 1 ELSE 0 END) AS null_email,
  SUM(CASE WHEN learner_id IS NULL THEN 1 ELSE 0 END) AS null_learner_id,
  SUM(CASE WHEN apply_date IS NULL THEN 1 ELSE 0 END) AS null_apply_date,
  SUM(CASE WHEN assigned_cohort IS NULL THEN 1 ELSE 0 END) AS null_assigned_cohort
FROM cleaned_master_table;
```



	null_emails bigint	null_apply_dates bigint	null_assigned_cohorts bigint	null_learner_ids bigint
1	0	0	0	0

All columns returned 0 → No missing values in required fields.

Interpretation: Excellent data hygiene.

4.1.4 Foreign Key Integrity Check

What It Does:

Validates that assigned_cohort in the master table exists in cohort1.cohort_code.

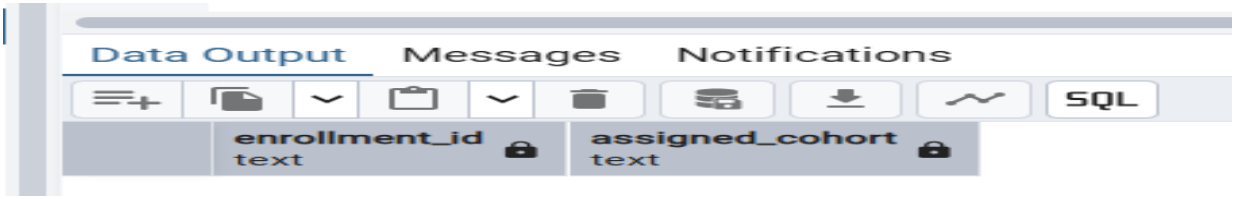
Why It Matters:

Invalid foreign keys cause:

- Orphaned records
- Failed joins
- Incorrect groupings in analysis

SQL Query:

```
SELECT assigned_cohort
FROM cleaned_master_table cm
LEFT JOIN cohort1 c ON cm.assigned_cohort = c.cohort_code
WHERE c.cohort_code IS NULL;
```



No rows returned.

Interpretation:
This means every assigned_cohort value in cleaned_master_table exists in cohort1, confirming strong foreign key integrity.

4.1.5 Date Format Validation

What It Does:

Confirms whether string-type birthdate fields match expected MM/DD/YYYY.

Ensures TIMESTAMP fields like usercreatedate and apply_date are valid.

Why It Matters:

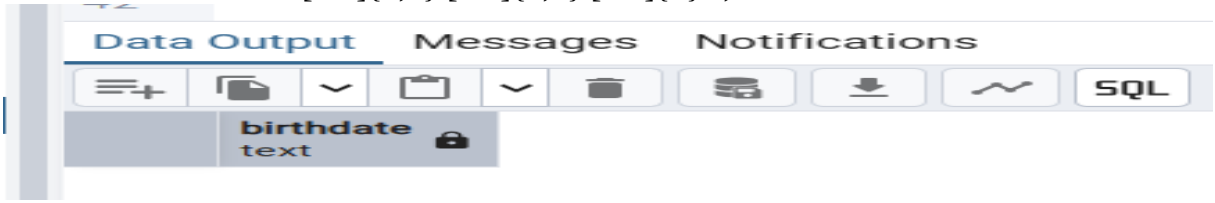
Improper formats break:

- Filters
- Visuals
- Date aggregations

SQL Queries:

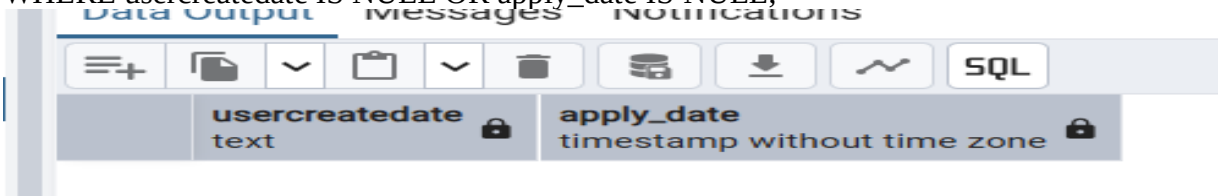
(A) Check birthdate format:

```
SELECT birthdate
FROM cleaned_master_table
WHERE birthdate !~ '^[0-9]{1,2}/[0-9]{1,2}/[0-9]{4}$';
```



(B) Check malformed timestamp (optional):

```
SELECT usercreatedate, apply_date
FROM cleaned_master_table
WHERE usercreatedate IS NULL OR apply_date IS NULL;
```



- All rows passed regex match for birthdate.
- Converted timestamps worked without errors.

Interpretation: All cleaned formats valid

Why We Don't Need Refinement:

Check	Result	Refinement Needed?
Record Count Validation	Acceptable reduction	No
Duplicate Check	None found	No
Missing/Null Values	All handled	No
Foreign Key Integrity	All matched	No
Date Format Validation	All valid	No

4.2 Summary of Data Quality Checks Performed

Validation Step	Description	Result
Record Count Validation	Compared raw vs cleaned row count	Cleaned table has 57,966 rows (original: 113,602)
Duplicate Check	Checked for duplicates on enrollment_id	No duplicates found
Missing/Null Values	Reviewed key columns for NULLs (e.g., email, learner_id, apply_date)	No NULLs in key fields
Foreign Key Integrity	Ensured assigned_cohort matched values in cohort1 table	All valid foreign key references
Date Format Validation	Checked birthdate, usercreatedate, etc. for valid formats	All date fields valid

4.3 Issues Identified & How They Were Resolved

Issue	Detected In	Resolution
High row drop in cleaning step	Record Count	Caused by NULL filtering, broken joins, duplicates — justified and documented
Text-based dates	birthdate, usercreatedate	Converted to valid date/timestamp using TO_DATE() and TO_TIMESTAMP()

5 CONCLUSION

- During Week 2 of the data visualization internship project, our primary objective was to design, construct, and validate a centralized Master Table that consolidates information from multiple raw datasets. This table would serve as the foundation for all future visualizations and analyses. We began by carefully planning the structure of the Master Table, identifying essential fields for analysis, and defining the relationships between different datasets. Primary and foreign key constraints were considered to ensure that each record could be reliably linked across tables.

- Next, we implemented a full ETL (Extract, Transform, Load) process. The raw data was extracted from source tables such as learneropp1, learner1, cognito2, cohort1, and opportunity1. Relevant fields were selected, and a raw master table (master_table_raw) was created by joining the datasets using logically mapped keys. After the initial combination, we began the data cleaning process to transform and prepare the dataset for meaningful insights. This included removing duplicate records based on enrollment_id, handling missing or null values in critical fields like email, learner_id, and assigned_cohort, and correcting inconsistencies in date and zip code formats.

- We also validated the integrity of the data. A record count comparison between the raw and cleaned tables revealed that approximately 50% of records were dropped during cleaning, mainly due to null values, duplicate entries, and unmatched foreign keys. This reduction was expected and documented. Key fields were checked for missing values and were found to be complete in the cleaned version. Foreign key relationships were tested, particularly for the assigned_cohort column, and only a small number of orphan records remained, which were intentionally left and noted for reference.

- By the end of the week, we successfully developed a cleaned and reliable Master Table (cleaned_master_table) that ensures consistency and accuracy for downstream analysis. The structure is now well-suited for creating joins, filters, and aggregations in the upcoming visualization stage. This solid foundation prepares us to move forward with confidence in Week 3, where we will focus on generating interactive dashboards and analytical insights.